4 Running Migrations

Rails provides a set of Rake tasks to run certain sets of migrations.

The very first migration related Rake task you will use will probably be `rake db:migrate`. In its most basic form it just runs the `change` or `up` method for all the migrations that have not yet been run. If there are no such migrations, it exits. It will run these migrations in order based on the date of the migration.
Note that running the `db:migrate` task also invokes the `db:schema:dump` task, which will update your `db/schema.rb` file to match the structure of your database.
If you specify a target version, Active Record will run the required migrations (change, up, down) until it has reached the specified version. The version is the numerical prefix on the migration's filename. For example, to migrate to version 20080906120000 run:

```
$ bin/rake db:migrate VERSION=20080906120000
```

If version 20080906120000 is greater than the current version (i.e., it is migrating upwards), this will run the `change` (or `up`) method on all migrations up to and including 20080906120000, and will not execute any later migrations. If migrating downwards, this will run the `down` method on all the migrations down to, but not including, 20080906120000.

4.1 Rolling Back

A common task is to rollback the last migration. For example, if you made a mistake in it and wish to correct it. Rather than tracking down the version number associated with the previous migration you can run:

```
$ bin/rake db:rollback
```

This will rollback the latest migration, either by reverting the `change` method or by running the `down`method. If you need to undo several migrations you can provide a `STEP` parameter:

```
$ bin/rake db:rollback STEP=3
```

will revert the last 3 migrations.

The `db:migrate:redo` task is a shortcut for doing a rollback and then migrating back up again. As with the `db:rollback` task, you can use the `STEP` parameter if you need to go more than one version back, for example:

```
$ bin/rake db:migrate:redo STEP=3
```

Neither of these Rake tasks do anything you could not do with `db:migrate`. They are simply more convenient, since you do not need to explicitly specify the version to migrate to.

4.2 Setup the Database

The `rake db:setup` task will create the database, load the schema and initialize it with the seed data.

4.3 Resetting the Database

The `rake db:reset` task will drop the database and set it up again. This is functionally equivalent to `rake db:drop db:setup`.
This is not the same as running all the migrations. It will only use the contents of the current`schema.rb` file.
If a migration can't be rolled back, `rake db:reset` may not help you. To find out more about dumping the schema see Schema Dumping and You section.